Adaptalyfe — Payment Guide: Subscription SaaS vs Ecommerce + E2E Integration

One document for:

1. **All common business models** + clear **Adaptalyfe recommendation**
2. **Subscription vs ecommerce** (and freemium, usage, license, marketplace, hybrid)
3. **End-to-end integration** (Stripe, Apple, Google Play, keys, plan IDs)

Related:

- `mobile_flutter/GOOGLE_PLAY_SUBSCRIPTIONS_SETUP.md`
- `server/stripe-plan-prices.ts` — fixed Stripe Price IDs

Part A — Business models (all options) + Adaptalyfe recommendation

A1. All common models (overview)

#	Model	How the customer pays	What you manage	Good fit when…
1	**Subscription SaaS**	Monthly / yearly plan	Plan keys, Price / store product IDs, entitlement on user	Ongoing software access (apps, tools)
2	**Ecommerce**	One-time order	SKUs, inventory, orders, shipping	Physical or one-shot digital goods
3	**Freemium**	Free tier + paid upgrade	Free limits + paid plans	Grow users first, convert later
4	**Usage / metered**	Pay per seat, API call, minute, token	Meters, invoices, overages	Variable consumption (AI, SMS, cloud)
5	**One-time license**	Pay once, keep access	License key / perpetual entitlement	Desktop tools, courses, lifetime deals
6	**Marketplace**	Buyers pay sellers; platform takes a cut	Connected accounts, payouts, fees	Two-sided platforms
7	**Donation / tip**	Optional voluntary pay	Tips, no hard entitlement	Creators, nonprofits
8	**Hybrid**	Mix of above	Separate modules per revenue type	Plan + merch, or plan + usage add-ons

These are **different payment designs**. Do not mix cart/SKU logic with subscription entitlement unless you intentionally build a hybrid with clear boundaries.

A2. Model details

1) Subscription SaaS (product company)

- Sell **access** (Basic / Premium / Family), not inventory.
- Recurring Stripe Subscription (web) and/or Apple / Google IAP (mobile).
- Success: active subscribers, MRR, churn.
- Entitlement: `users.subscription_tier` + status.

- **This is Adaptalyfe's primary model.**

2) Ecommerce website

- Sell **items** via cart → Checkout / PaymentIntent.
- IDs: Stripe Product + Price (or Checkout), your `orders` table, SKUs.
- Usually **no** Apple/Play subscription IDs for physical goods.
- Success: orders, conversion, AOV.
- Optional: membership add-on (then you add a small subscription slice).

3) Freemium

- Free account with limits; paid plans unlock more.
- Adaptalyfe already has a **free / inactive** default plus paid tiers and a **7-day trial**.
- Works **together with** subscription SaaS (not instead of it).

4) Usage / metered

- Bill by usage (tasks over limit, AI tokens, caregiver seats beyond plan).
- Stripe metered prices or invoice items.
- Heavier ops; only add if product needs overage billing.
- Can sit **on top of** a base subscription (hybrid).

5) One-time license

- Pay once for lifetime (or long-term) access.
- Simpler than subscriptions; no renewals.
- Poor fit for ongoing care/support apps that need continuous updates and cloud cost.

6) Marketplace

- Platform connects buyers and sellers; takes a fee (Stripe Connect).
- Not Adaptalyfe's current product (no multi-vendor payouts).

7) Donation / tip

- Optional support payments.
- Not a substitute for plan entitlement.

8) Hybrid

- Example: **Subscription** (core) + **Ecommerce** (merch kits) + optional **usage** add-ons.
- Keep modules separate: plan entitlement ≠ shop orders.

A3. Subscription vs ecommerce (core difference)

Topic	Subscription SaaS	Ecommerce shop
What you sell	Access plans	Goods / one-time items
Customer pays	Recurring monthly / yearly	Usually one-time per order
Main IDs	Plan key + Price / store product IDs	SKU + Stripe Product/Price + order id
Card on file	Yes (renewals)	Optional
Apple / Google IAP	Required for in-app digital subscriptions	Rare for physical goods
Success metric	Subscribers / MRR	Orders / AOV
Entitlement	User subscription fields	Order line items / inventory
Cancel	Stop renewal; access until period end	Refund / return

Stripe objects	Customer + **Subscription** + Price	Customer + **PaymentIntent** / Checkout + Product

---

A4. IDs by model

Ecommerce-only website

ID	Example	Purpose
Product	`prod_…`	Item in Stripe
Price	`price_…`	Amount
SKU / metadata	your SKU	Map cart → Stripe
Order id	`orders.id`	Fulfillment

Flow: Cart → Checkout → webhook → mark paid → ship/deliver.

Subscription product company (Adaptalyfe)

Layer	IDs
Business plan	`basic` / `premium` / `family`
Billing cycle	`monthly` / `annual`
Stripe (web)	6× fixed `price_…` in env
Apple	6× `adaptalyfe_*_{monthly|annual}`
Google Play	3× `adaptalyfe_*_monthly` today

Entitlement is on the **user**, not a cart order.

A5. Adaptalyfe product recommendation (use this)

Primary recommendation

Use: Subscription SaaS + Freemium (plan-based product company).

Choose	Why for Adaptalyfe
Subscription SaaS	Core product is ongoing life-support software (tasks, caregivers, health) — needs recurring access and updates
Freemium / trial	Already in product: free default + 7-day trial + Basic / Premium / Family
Monthly + yearly	Matches landing + Stripe + Apple; Play monthly first
Fixed Stripe Price IDs	Pro ops; stops duplicate products (`STRIPE_PRICE_*`)
Store IAP on mobile	Required for digital subscriptions in iOS/Android apps

Do **not** make primary

Avoid as core	Why
Pure ecommerce cart for plans	Plans are entitlements, not inventory SKUs

Marketplace	No seller payouts in current product
Donation-only	Cannot fund reliable cloud + support
One-time license only	Ongoing server/AI/support costs need renewals
Usage-only billing	Too complex as the only model; optional later as add-on

Optional later (hybrid, separate modules)

1. **Ecommerce** — merch / physical kits (own `orders` flow; do not change `subscription_tier`).
2. **Usage / metered** — overages (extra seats, AI) on top of a base plan.
3. **Family seat packs** — still subscription/entitlement, not a shopping cart.

One-line decision

> ***Adaptalyfe = subscription product company (Basic / Premium / Family × monthly / yearly), with freemium/trial; ecommerce or usage only as separate add-ons later.***

Part B — What Adaptalyfe has today (E2E)

B1. Three payment integrations

#	Platform	Surface	Plans	Cycles in code
1	**Stripe**	Website	basic, premium, family	Monthly + Annual
2	**Apple App Store**	iOS	basic, premium, family	Monthly + Annual
3	**Google Play**	Android	basic, premium, family	**Monthly only** (annual not wired yet)

Plan key	Display	Monthly	Annual
`basic`	Basic	**$4.99** (499¢)	**$49.99** (4999¢)
`premium`	Premium	**$12.99** (1299¢)	**$129.99** (12999¢)
`family`	Family / Care Team	**$24.99** (2499¢)	**$249.99** (24999¢)

- Free trial: **7 days** (`FREE_TRIAL_DAYS`).
- Package / bundle: **`com.adaptalyfe.app`**.

B2. Preferred catalog IDs

API cycle name: `monthly` | `annual`
(Landing may say “yearly” → map to **`annual`** for Stripe.)

Plan	Monthly product ID	Annual product ID
Basic	`adaptalyfe_basic_monthly`	`adaptalyfe_basic_annual`
Premium	`adaptalyfe_premium_monthly`	`adaptalyfe_premium_annual`
Family	`adaptalyfe_family_monthly`	`adaptalyfe_family_annual`

Stripe metadata `plan_key`: same strings (e.g. `adaptalyfe_premium_annual`).

B3. Entitlement (source of truth)

Table: `users`

Column	Meaning
`subscription_tier`	`free` / `basic` / `premium` / `family`
`subscription_status`	`active`, `pending`, `trialing`, `past_due`, `cancelled`, `expired`, …
`subscription_platform`	`web` / `app_store` / `google_play`
`subscription_expires_at` / `subscription_start_date`	Period
`stripe_customer_id` / `stripe_subscription_id`	Stripe
`google_play_*` / `apple_original_transaction_id`	Stores

App reads: `GET /api/subscription` → plan, status, features, limits.

(Ecommerce would use `orders` / line items instead — we do not use that for core plan access.)

Do we need database column changes?

No — not for the current subscription + fixed Stripe Price ID work.

Change	Need new DB column?	Where it lives instead
Fixed Stripe Price IDs (`STRIPE_PRICE_*`)	**No**	Environment variables only
Company API keys (Stripe / Apple / Play)	**No**	Environment / secrets
Plan Basic / Premium / Family	**No**	Existing `subscription_tier`
Monthly / annual	**No**	Encoded in Price / store product id; not a required user column
Per-user Stripe customer	**No**	Existing `stripe_customer_id`
Per-user Stripe subscription	**No**	Existing `stripe_subscription_id`
Apple / Play purchases	**No**	Existing Apple / Play columns on `users`

`users` already has the entitlement columns the app uses. The `subscriptions` table exists in schema but **runtime entitlement uses `users`**, not that table.

Only add columns/tables later if you build something new, for example:

Future feature	Possible DB work
Ecommerce merch shop	New `orders` / `order_items` tables
Usage / metered billing	Usage counters or meter events table
Store Stripe Price id on user for audit	Optional `stripe_price_id` on `users` (nice-to-have, not required)

For today's pro setup (plans + env Price IDs + keys): **schema is enough — no migration required.**

B4. Flows

Website (Stripe) — subscription, not cart ecommerce

```
Landing Choose plan → register → /subscription
  → POST /api/create-subscription { planType, billingCycle }
  → Stripe Subscription + trial + SetupIntent/PaymentIntent
  → POST /api/confirm-subscription
  → users updated (platform = web)
  → Stripe webhooks renew / cancel / past_due
  → GET /api/subscription
```

iOS (Apple)

```
StoreKit → POST /api/apple/verify-purchase → users (app_store)
ASN V2 → POST /api/apple/notifications
Restore → POST /api/apple/restore-purchases
```

Android (Google Play)

```
Play Billing → POST /api/google-play/verify-purchase → users (google_play)
RTDN → POST /api/google-play/notifications
Restore → POST /api/google-play/restore-purchases
```

B5. API map

Method	Path	Role
GET	`/api/subscription`	Entitlement
POST	`/api/create-subscription`	Stripe start
POST	`/api/confirm-subscription`	Stripe activate
POST	`/api/recover-subscription`	Stripe re-sync
POST	`/api/stripe/webhook`	Stripe lifecycle
POST	`/api/apple/verify-purchase`	iOS
POST	`/api/apple/restore-purchases`	iOS
POST	`/api/apple/notifications`	ASN V2
POST	`/api/google-play/verify-purchase`	Android
POST	`/api/google-play/restore-purchases`	Android
POST	`/api/google-play/notifications`	RTDN
POST	`/api/create-payment-intent`	Legacy one-shot (not primary plan path)
POST	`/api/upgrade-subscription`	Retired **410**
POST	`/api/subscription/upgrade`	Blocked **409**

B6. Key management (how it works — company keys vs per-user IDs)

Important rule

Type	Same for all customers?	Where stored	Example
Company / platform keys	**Yes** — one set per environment (staging or production)	Server env (Railway / `.env`)	`STRIPE_SECRET_KEY`, Apple `.p8`, Play service account
Catalog Price / Product IDs	**Yes** — shared catalog	Env + Stripe/App Store/Play Console	`STRIPE_PRICE_PREMIUM_MONTHLY=price_…`
Customer / user billing IDs	**No — different per user**	Database `users` row	`stripe_customer_id`, `stripe_subscription_id`, Apple original transaction, Play purchase token
Login session token	**No** — different per login/device	Cookie / localStorage / mobile secure storage	Bearer session after `/api/login`

You do **not** give each end-user their own Stripe secret key.
You give each end-user their own **Customer / Subscription / purchase** IDs in the database.

```
Company (Adaptalyfe)
  └── Staging env keys (test)     ← shared by all staging users
  └── Production env keys (live)  ← shared by all live users

Each user row in DB
  └── User A: stripe_customer_id = cus_AAA, subscription = sub_AAA, tier = premium
  └── User B: stripe_customer_id = cus_BBB, subscription = sub_BBB, tier = basic
  └── User C: apple_original_transaction_id = 1000000… (no Stripe customer)
```

Environments (where company keys live)

Level	Stripe	Apple	Play
Local	**test** keys	sandbox	license testers
Staging (`ai-staging`)	**test** keys	sandbox	internal testing
Production (`main`)	**live** keys only	production	production

Never mix: live Stripe secret on staging, or test prices pointed at live keys.

A) Company keys (shared — NOT per customer)

These unlock Stripe / Apple / Google APIs for the **whole app**. One secret per environment.

Stripe (shared)

Variable	Role	Per user?
`STRIPE_SECRET_KEY`	Server charges / creates subscriptions (`sk_test_…` or `sk_live_…`)	**No** — company key
`VITE_STRIPE_PUBLIC_KEY`	Browser Stripe.js (`pk_test_…` / `pk_live_…`)	**No** — public publishable key

`STRIPE_WEBHOOK_SECRET`	Verify webhook signatures (`whsec_…`)	**No**
`STRIPE_PRICE_BASIC_MONTHLY` … `STRIPE_PRICE_FAMILY_ANNUAL`	Which Price to sell	**No** — catalog (6 shared IDs)
`STRIPE_TEST_CLOCK_ID`	Optional test renewals	**No**

Example (shape only — do not commit real secrets):

```
# Staging / local (TEST mode)
STRIPE_SECRET_KEY=sk_test_51xxxxxxxx
VITE_STRIPE_PUBLIC_KEY=pk_test_51xxxxxxxx
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxx
STRIPE_PRICE_BASIC_MONTHLY=price_1BasicMonthlyTest
STRIPE_PRICE_BASIC_ANNUAL=price_1BasicAnnualTest
STRIPE_PRICE_PREMIUM_MONTHLY=price_1PremMonthlyTest
STRIPE_PRICE_PREMIUM_ANNUAL=price_1PremAnnualTest
STRIPE_PRICE_FAMILY_MONTHLY=price_1FamMonthlyTest
STRIPE_PRICE_FAMILY_ANNUAL=price_1FamAnnualTest

# Production (LIVE mode) — different keys + different price_ IDs
STRIPE_SECRET_KEY=sk_live_51xxxxxxxx
VITE_STRIPE_PUBLIC_KEY=pk_live_51xxxxxxxx
STRIPE_WEBHOOK_SECRET=whsec_live_xxxxxxxx
STRIPE_PRICE_BASIC_MONTHLY=price_1BasicMonthlyLive
# …same six vars, live Price IDs
```

Production: missing `STRIPE_PRICE_*` → checkout error (**no** auto-create Prices).
Dev: may fall back to metadata search/create.

Apple (shared)

Variable	Per user?
`APP_STORE_CONNECT_ISSUER_ID`	No
`APP_STORE_CONNECT_KEY_ID`	No
`APP_STORE_CONNECT_PRIVATE_KEY`	No (company `.p8`)
`APPLE_BUNDLE_ID` / `APPLE_APP_ID`	No

Google Play (shared)

Variable	Per user?
`GOOGLE_PLAY_SERVICE_ACCOUNT_KEY`	No (company JSON)
`GOOGLE_PLAY_PUBSUB_AUDIENCE`	No
`GOOGLE_PLAY_PUBSUB_SERVICE_ACCOUNT_EMAIL`	No

Package: `com.adaptalyfe.app` (hardcoded).

Never commit `.env`, `.p8`, or service-account JSON. Store them only in Railway/hosting secrets (and local `.env` gitignored).

B) Per-user / per-customer IDs (different for every user)

Created when that person pays or restores. Stored on **`users`**.

Field on `users`	Who has it	Meaning
`stripe_customer_id`	Web subscribers	Stripe Customer `cus_…` — **unique per user**
`stripe_subscription_id`	Web subscribers	Stripe Subscription `sub_…` — **unique per user**
`subscription_tier`	Everyone	`free` / `basic` / `premium` / `family`
`subscription_status`	Everyone	`active` / `pending` / …
`subscription_platform`	Paid users	`web` / `app_store` / `google_play`
`apple_original_transaction_id`	iOS	Apple's long-lived purchase id — **per Apple account / purchase**
`google_play_purchase_token`	Android	Play token — **per purchase**
`google_play_product_id` / `subscription_product_id`	Mobile	Which store product they bought

Example after three different customers subscribe:

User	Platform	Customer / purchase ID	Plan
Soham (id 98)	web	`cus_Soham…` + `sub_Soham…`	premium
Mit (id 12)	web	`cus_Mit…` + `sub_Mit…`	basic
Aiden (id 41)	app_store	`apple_original_transaction_id=2000000…`	family

Same company `STRIPE_SECRET_KEY`, but **three different** customer/subscription IDs.

How they get created:

1. User logs in (session is per user).
2. `POST /api/create-subscription` → Stripe creates/retrieves **that user's** `cus_…`, then `sub_…`.
3. Server saves IDs on **that** `users` row.
4. `GET /api/subscription` reads **that** row (and may refresh Play/Apple for that user only).

C) What is NOT per-user

Item	Shared?	Why
`STRIPE_SECRET_KEY`	Shared	Company account with Stripe
`price_…` Basic/Premium/Family	Shared	Same catalog for everyone
`adaptalyfe_premium_monthly` product id	Shared	Same App Store / Play product
Landing "Choose plan" prices	Shared	Display catalog

Item	Per user?	Why
Who is on Basic vs Premium	Yes	Entitlement
`cus_` / `sub_` / purchase token	Yes	Their billing contract
Card last4 / payment method	Yes (in Stripe Customer)	Their card — we don't store full PAN

D) Practical management checklist

1. **Create 2 Stripe accounts/modes:** Test + Live (or one account with Test/Live toggles).
2. **Put company keys only in env** for that environment — not in the database, not in git.
3. **Put 6 Price IDs in env** (staging test prices / production live prices).
4. **Never create a new Stripe secret per customer** — create a Customer object instead.
5. **On signup/subscribe**, persist `stripe_customer_id` (and later `stripe_subscription_id`) on that user.
6. **On mobile verify**, persist Apple/Play tokens on that user.
7. **Support lookups:** find user by email → read their `cus_` / `sub_` / store ids → check Stripe or store dashboard.
8. **Rotate company keys** if leaked; user Customer IDs stay the same in Stripe.
9. Rebuild frontend after changing `VITE_STRIPE_PUBLIC_KEY`.

E) Ecommerce note (if you add a shop later)

Same pattern:

- **Shared:** Stripe secret + product/price catalog.
- **Per customer:** `cus_…` + **order ids** (`ord_…` / your `orders.id`) — not a new secret key per shopper.

B7. Stripe dashboard checklist (plan catalog)

Per mode (Test, then Live):

1. Keep **6 Prices** (3 plans × monthly/annual) at the amounts above.
2. Metadata `plan_key = adaptalyfe_{plan}_{cycle}`.
3. Copy `price_…` → `STRIPE_PRICE_*` env.
4. Webhook → `/api/stripe/webhook`.
5. Archive duplicate Basic/Premium/Family products.
6. Redeploy.

B8. Feature access (subscription plans)

Capability	Basic / trial	Premium	Family
Core tasks / mood / calendar	Yes	Yes	Yes
Wearables, meals, meds, advanced analytics	No	Yes	Yes
Family dashboard / multi-user / location safety	No	No	Yes

Exact flags: `server/subscription-response.ts`, `server/subscription-access.ts`.

Part C — Decision sheet (product company)

Question	Answer for Adaptalyfe
Recommended primary model	**Subscription SaaS + Freemium / trial**
Not primary	Ecommerce-only, marketplace, donation-only, one-time license-only
Catalog	**Basic / Premium / Family × monthly / yearly**
Do we need cart/SKU for core plans?	**No**
Minimum IDs for web-only?	3 plans × 2 cycles → **6 Stripe Price IDs**
Why more IDs?	iOS + Android require **store product IDs**

Prefer fixed Stripe Prices?	**Yes** (`STRIPE_PRICE_*`)
Optional later	Separate **ecommerce** (merch) or **usage** add-ons — never mixed into plan entitlement
Official recommendation	See **Part A5**

Code map

Area	Path
Stripe Price resolution	`server/stripe-plan-prices.ts`
Routes	`server/routes.ts`
Entitlement response	`server/subscription-response.ts`
Feature gates	`server/subscription-access.ts`
Apple / Play maps	`server/apple-subscription-entitlement.ts`, `server/google-play-entitlement.ts`
Landing / paywall	`client/src/pages/landing.tsx`, `subscription.tsx`
Schema	`shared/schema.ts` → `users`

This document covers both models (subscription vs ecommerce), how they differ, and the Adaptalyfe E2E payment setup we prefer for a product company.